

ใบความรู้ที่ 8 การประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP)

วัตถุประสงค์การเรียนรู้

- เข้าใจความหมาย หลักการทำงาน และข้อจำกัดทางสถาปัตยกรรมของการประมวลผลภาษาธรรมชาติ (NLP)
- เข้าใจความท้าทายเชิงโครงสร้างของภาษาไทยในการตัดคำ (Tokenization) และหลักการทำงานของอัลกอริทึมตัดคำแบบการเรียนรู้เชิงลึก
- เข้าใจหลักการทางคณิตศาสตร์ของการแปลงคำศัพท์เป็นปริภูมิเวกเตอร์ (Word Embedding) ด้วยสถาปัตยกรรม Continuous Bag-of-Words (CBOW) และ Skip-Gram
- เข้าใจโครงสร้างและกลไกประตูควบคุม (Gates) ในเซลล์หน่วยความจำระยะสั้นแบบยาว (LSTM) รวมถึงสถาปัตยกรรมแบบสองทิศทาง (Bidirectional)
- รู้วิธีการทำวิศวกรรมข้อมูลข้อความ (Data Engineering) ตั้งแต่การทำความสะอาดข้อมูล และการปรับความยาวประโยค (Sequence Padding) เพื่อเตรียมความพร้อมสำหรับโมเดลปัญญาประดิษฐ์
- เข้าใจหลักเกณฑ์การประเมินประสิทธิภาพของระบบวิเคราะห์อารมณ์ความรู้สึกผ่านตารางความสับสน (Confusion Matrix) และมาตรวัด F1-Score
- รู้จักแนวทางการประยุกต์ใช้เทคโนโลยีวิเคราะห์อารมณ์ความรู้สึก (Sentiment Analysis) ในสถานการณ์จริง และตระหนักถึงอคติที่อาจแฝงอยู่ในชุดข้อมูล

1. ปรัชญาและทฤษฎีรากฐานของการประมวลผลภาษาธรรมชาติ

การประมวลผลภาษาธรรมชาติ (Natural Language Processing หรือ NLP) เป็นแขนงวิชาหลักภายใต้ร่มใหญ่ของปัญญาประดิษฐ์ (Artificial Intelligence) ที่ทำหน้าที่เป็นสะพานเชื่อมระหว่างการสื่อสารของมนุษย์ซึ่งมีความซับซ้อน ไร้โครงสร้างที่ตายตัว และตรรกะทางคณิตศาสตร์ของเครื่องจักรคอมพิวเตอร์ เป้าหมายสูงสุดของ NLP ไม่ใช่เพียงการทำให้คอมพิวเตอร์ "อ่าน" ตัวอักษรออก แต่เป็นการทำให้คอมพิวเตอร์ "ทำความเข้าใจ" บริบท นัยยะแฝง และอารมณ์ความรู้สึกที่ผู้พูดหรือผู้เขียนต้องการสื่อสาร

ในเชิงเปรียบเทียบ การทำให้คอมพิวเตอร์ประมวลผลรูปภาพ (Computer Vision) นั้นมีความตรงไปตรงมาทางคณิตศาสตร์มากกว่า เนื่องจากภาพประกอบด้วยจุดพิกเซล (Pixels) ซึ่งแต่ละจุดมีค่าตัวเลขที่ชัดเจน (เช่น ค่าความเข้มแสงตั้งแต่ 0 ถึง 255 ในระบบสี RGB) โครงข่ายประสาทเทียมสามารถเรียนรู้รูปแบบจากแสงและขอบเขตของสีได้อย่างเป็นรูปธรรม ในทางตรงกันข้าม "ข้อความ" หรือ "ประโยค"

ไม่มีมาตรวัดเชิงปริมาณในตัวเอง คอมพิวเตอร์มองเห็นตัวอักษรเป็นเพียงรหัสมาตรฐาน เช่น ASCII หรือ Unicode ซึ่งไม่ได้สื่อถึงความหมายทางภาษาศาสตร์ใด ๆ ตัวอย่างเช่น รหัสของคำว่า "รัก" และ "เกลียด" ในระบบคอมพิวเตอร์ไม่ได้มีระยะห่างทางคณิตศาสตร์ที่บ่งบอกถึงอารมณ์ที่ขัดแย้งกันเลย

ยิ่งไปกว่านั้น ภาษามีลักษณะเชิงลำดับเวลา (Temporal or Sequential Structure) ที่มีความสำคัญอย่างยิ่งยวด การเรียงลำดับคำที่ต่างกันเพียงเล็กน้อยสามารถเปลี่ยนความหมายของประโยคไปอย่างสิ้นเชิง ตัวอย่างที่เห็นได้ชัดคือประโยค "สุนัขกัดคน" กับ "คนกัดสุนัข" ซึ่งใช้ชุดคำศัพท์ (Vocabulary) ชุดเดียวกันทุกประการ ทว่าบริบทและความหมายกลับแตกต่างกันอย่างสิ้นเชิง นอกจากนี้ การเติมคำปฏิเสธเพียงคำเดียว เช่น การเปลี่ยนจาก "อร่อย" เป็น "ไม่อร่อย" ย่อมส่งผลให้ความหมายเชิงอารมณ์พลิกกลับจากขี้บวกลบเป็นขี้ลบลบทันที ความเปราะบางและความละเอียดอ่อนเหล่านี้ทำให้การสร้างระบบ NLP จำเป็นต้องอาศัยเทคนิคทางวิศวกรรมข้อมูลและสถาปัตยกรรมโครงข่ายประสาทเทียมที่ออกแบบมาเพื่อจัดการกับ "บริบทเชิงลำดับ" โดยเฉพาะ

1.1. กลไกการตัดคำ (Tokenization) และความท้าทายในระบบภาษาไทย

ก้าวแรกในการแปลงข้อความภาษามนุษย์ให้เป็นข้อมูลที่เครื่องจักรสามารถประมวลผลได้คือกระบวนการ "การตัดคำ" (Tokenization) ซึ่งหมายถึงการแบ่งส่วนประโยคหรือข้อความที่มีความยาวออกเป็นหน่วยย่อยที่สุดที่มีความหมายในตัวเอง (Tokens) ซึ่งอาจเป็นคำ (Words) พยางค์ (Syllables) หรือแม้กระทั่งอักขระย่อย (Sub-words)

สำหรับภาษาตระกูลอินโด-ยูโรเปียน เช่น ภาษาอังกฤษ กระบวนการ Tokenization สามารถดำเนินการได้อย่างตรงไปตรงมาและใช้ทรัพยากรการคำนวณต่ำ เนื่องจากโครงสร้างการเขียนภาษาอังกฤษมีการใช้สัญลักษณ์ช่องว่าง (Spacebar) เพื่อแบ่งขอบเขตของคำ (Word Boundary) อย่างชัดเจน คอมพิวเตอร์จึงสามารถใช้อัลกอริทึมการแยกสตริงพื้นฐาน (String Splitting) เพื่อแบ่งประโยคออกเป็นคำย่อย ๆ ได้ทันที

อย่างไรก็ตาม ภาษาไทยถูกจัดอยู่ในกลุ่มภาษาที่เขียนแบบต่อเนื่อง (Unsegmented Language) โดยไม่มีเครื่องหมายหรือช่องว่างในการแบ่งแยกคำอย่างชัดเจน โครงสร้างนี้ก่อให้เกิดความท้าทายที่สำคัญที่สุดประการหนึ่งในวงการ NLP ของไทย นั่นคือปัญหา "ความกำกวมในการแบ่งขอบเขตคำ" (Word Boundary Ambiguity) ตัวอย่างคลาสสิกของปัญหานี้คือลำดับอักขระ "ตากลม" ซึ่งระบบคอมพิวเตอร์จำเป็นต้องตัดสินใจว่าจะตัดคำเป็น "ตา-กลม" (ดวงตาที่มีลักษณะกลม) หรือ "ตาก-ลม" (การผึ่งลม) การตัดสินใจที่ถูกต้องไม่สามารถทำได้โดยการพิจารณาเพียงลำดับอักขระเดี่ยว ๆ แต่ต้องอาศัยการตีความจากบริบทแวดล้อมทั้งหมดของประโยคนั้น

วิวัฒนาการของการแก้ปัญหาการตัดคำภาษาไทยได้ถูกพัฒนาอย่างต่อเนื่อง ในยุคแรก ระบบมักพึ่งพาการจับคู่กับพจนานุกรม (Dictionary-based Approach) และกฎเกณฑ์เชิงไวยากรณ์ (Rule-based) โดยใช้อัลกอริทึมแบบละโมภ (Greedy Algorithm) หรือการค้นหาคำที่ยาวที่สุด (Longest Matching) ทว่าวิธีการเหล่านี้มีข้อจำกัดอย่างร้ายแรงเมื่อต้องเผชิญกับชุดข้อมูลในโลกความเป็นจริง โดยเฉพาะอย่างยิ่งข้อมูล

จากโซเซียมมีเดียที่มีการใช้ศัพท์แสลง คำทับศัพท์ การสะกดคำผิด และคำศัพท์เกิดใหม่ (Neologisms) หรือที่รวมเรียกว่าปัญหา "คำนอกพจนานุกรม" (Out-Of-Vocabulary: OOV)

ในปัจจุบัน การวิจัยและการประยุกต์ใช้งานจริงได้เปลี่ยนผ่านเข้าสู่ยุคของการใช้การเรียนรู้เชิงลึก (Deep Learning) สำหรับการตัดคำ เครื่องมือและไลบรารีสมัยใหม่ เช่น Deepcut ได้ถูกนำมาใช้เพื่อแก้ปัญหาได้อย่างแพร่หลาย ไลบรารีเหล่านี้นำเสนอการใช้สถาปัตยกรรมโครงข่ายประสาทเทียมแบบหลายชั้น (Multi-layer Neural Networks) เพื่อประเมินความน่าจะเป็นของรอยต่อระหว่างอักขระ (Character-level boundary probability) กลไกนี้ช่วยให้ระบบสามารถคาดเดาจุดตัดคำได้อย่างแม่นยำแม้จะต้องเผชิญกับคำศัพท์ที่ไม่มีอยู่ในระบบพจนานุกรมดั้งเดิมก็ตาม การตัดคำที่แม่นยำจึงเปรียบเสมือนการสร้างรากฐานที่มั่นคง หากการตัดคำในขั้นตอนนี้มีผิดพลาด ความหมายทั้งหมดที่โมเดลภาษาในระดับสูงกว่าจะประมวลผลย่อมผิดเพี้ยนตามไปด้วย

1.2. การแปลงคำเป็นเวกเตอร์ปริภูมิความหมาย (Word Embedding)

ภายหลังจากที่ประโยคถูกสับย่อยจนกลายเป็นคลังคำศัพท์ (Vocabulary/Corpus) สิ่งที่ต้องดำเนินการต่อไปคือขั้นตอนการแปลงคำศัพท์ (Vectorization) ให้อยู่ในรูปตัวเลขทางคณิตศาสตร์ ในยุคเริ่มต้นของ Machine Learning มีการใช้วิธีที่เรียกว่า One-Hot Encoding ซึ่งจะแปลงคำศัพท์แต่ละคำให้เป็นอาร์เรย์ของตัวเลข 0 และมีตัวเลข 1 อยู่เพียงตำแหน่งเดียวที่ตรงกับรหัส (Index) ของคำนั้น ๆ วิธีนี้แม้จะสามารถนำไปใช้กับโมเดลการเรียนรู้พื้นฐานได้ แต่ก็มีข้อบกพร่องทางสถาปัตยกรรมที่รุนแรงสองประการคือ:

1.2.1. ปัญหาความกระจัดกระจายของเมทริกซ์ (Sparsity Problem): หากพจนานุกรมมีคำศัพท์ 100,000 คำ เวกเตอร์ของคำศัพท์แต่ละคำก็จะมีขนาด 100,000 มิติ โดยมีเลข 0 ถึง 99,999 ตัว และเลข 1 เพียงตัวเดียว การคำนวณเมทริกซ์ขนาดใหญ่นี้สร้างภาระทางหน่วยความจำอย่างมหาศาล

1.2.2. การสูญเสียมิติทางความหมาย (Semantic Loss): เวกเตอร์ที่สร้างจาก One-Hot Encoding ทุกตัวจะมีคุณสมบัติทางคณิตศาสตร์คือการตั้งฉากซึ่งกันและกัน (Orthogonal) ทำให้ค่าความคล้ายคลึงโคไซน์ (Cosine Similarity) ระหว่างคำสองคำมีค่าเป็น 0 เสมอ ระบบจึงไม่สามารถรับรู้ได้เลยว่าคำว่า "ราชา" (King) มีความเกี่ยวข้องอย่างลึกซึ้งกับคำว่า "ราชินี" (Queen)

แนวทางปฏิบัติยุคใหม่จึงได้ปฏิวัติการแทนค่าคำศัพท์เข้าสู่กระบวนการที่เรียกว่า Word Embedding หลักการทางปรัชญาเบื้องหลังสถาปัตยกรรมนี้คือ "สมมติฐานการกระจายเชิงความหมาย" (Distributional Hypothesis) ซึ่งกล่าวไว้ว่า "คำศัพท์ที่ปรากฏอยู่ในบริบทแวดล้อมที่คล้ายคลึงกัน มักจะมีความหมายที่ใกล้เคียงกัน" Word Embedding ทำการบีบอัดมิติของพจนานุกรมจากหลักแสนมิติให้ลดลงเหลือเพียงปริภูมิเวกเตอร์แบบต่อเนื่อง (Continuous Vector Space) ที่มีความหนาแน่นสูง (Dense Vectors) ในมิติที่ต่ำกว่ามาก (โดยทั่วไปอยู่ระหว่าง 100 ถึง 300 มิติ) ในปริภูมินี้ คำศัพท์ที่มีความหมายคล้ายกันจะถูกจัดวางในพิกัดที่ใกล้เคียงกัน ทำให้เราสามารถนำเวกเตอร์ของคำศัพท์มาดำเนินการทางพีชคณิต

(Algebraic Operations) เพื่อสกัดความหมายเชิงตรรกะได้ เช่น การคำนวณสมการอันโด่งดัง:
Vector(King) – Vector(Man) + Vector(Woman) ≈ Vector(Queen)

หนึ่งในสถาปัตยกรรมรากฐานที่ได้รับความนิยมสูงสุดในการสร้าง Word Embedding คือ Word2Vec ซึ่งถูกคิดค้นโดยทีมวิจัยจาก Google อัลกอริทึมนี้ใช้โครงข่ายประสาทเทียมแบบตื้น (Shallow Neural Network) ที่มีเพียงชั้นซ่อน (Hidden Layer) เพียงชั้นเดียว โดยแบ่งออกเป็น 2 สถาปัตยกรรมย่อยที่ทำงานกลับทิศทางกัน ได้แก่ Continuous Bag-of-Words (CBOW) และ Skip-Gram :

มิติการวิเคราะห์	CONTINUOUS BAG-OF-WORDS (CBOW)	SKIP-GRAM
กลไกและทิศทางการทำนาย	ใช้กลุ่มคำบริบทรอบข้าง (Context Words) มาคำนวณหาค่าเฉลี่ยเพื่อทำนายผลคำเป้าหมายที่อยู่ตรงกลาง (Target Word)	ใช้คำเป้าหมายที่อยู่ตรงกลาง (Target Word) เพื่อทำนายคำแวดล้อม (Context Words) ที่กระจายอยู่โดยรอบภายในหน้าต่างที่กำหนด
จุดเด่นด้านประสิทธิภาพ	มีความเร็วในการเรียนรู้สูง เนื่องจากพิจารณาบริบททั้งหมดรวมกันเป็นก้อนเดียวเพื่อทำนายเพียงครั้งเดียว เหมาะสำหรับการจับโครงสร้างเชิงไวยากรณ์ (Syntactic) และคำศัพท์ที่ปรากฏบ่อย	ใช้เวลาในการฝึกฝนสูงกว่า เนื่องจากต้องทำนายข้อมูลหลายครั้งต่อหนึ่งคำเป้าหมาย แต่มีความละเอียดอ่อนสูงมากในการทำความเข้าใจความสัมพันธ์เชิงความหมาย (Semantic) ของคำที่หายาก (Rare Words)
ความเสี่ยงและการเรียนรู้เกินพอดี	มีโอกาสูงที่จะเกิดการเรียนรู้เกินพอดี (Overfitting) กับคำเชื่อมหรือคำศัพท์ทั่วไปที่พบได้บ่อยมากในฐานข้อมูล	มีความทนทานต่ออคติของคำศัพท์ที่พบบ่อยได้ดีกว่า เนื่องจากกลไกแยกการพิจารณาเป้าหมายในหลายมิติบริบท

ในบริบทของการวิเคราะห์อารมณ์ความรู้สึก (Sentiment Analysis) ซึ่งข้อความรวิวหรือความคิดเห็นมักประกอบด้วยคำวิเศษณ์ คำขยาย หรือศัพท์เฉพาะกลุ่มที่อาจไม่ได้ปรากฏบ่อยนัก (Rare Words) สถาปัตยกรรม **Skip-Gram** จึงได้รับการพิจารณาว่าเป็นตัวเลือกที่ทรงประสิทธิภาพมากกว่า โครงสร้างของ Skip-Gram ได้รับการออกแบบมาเพื่อเพิ่มค่าประมาณการความเป็นไปได้สูงสุดเชิงลอการิทึม (Maximum Log Probability) ระหว่างคำศูนย์กลางและคำแวดล้อมตามสมการ :

$$\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

โดยที่ T คือจำนวนคำศัพท์ทั้งหมดในฐานข้อมูล และ C คือขนาดของหน้าต่างบริบท (Context Window Size) ซึ่งสมการดังกล่าวสะท้อนให้เห็นว่าโมเดลพยายามจะปรับค่าน้ำหนักเวกเตอร์ให้สามารถอธิบายการปรากฏตัวร่วมกันของคำศัพท์ในระยะห่างที่กำหนดได้อย่างแม่นยำที่สุด

1.3. สถาปัตยกรรมโครงข่ายประสาทเทียมแบบมีความจำ (RNN และ LSTM)

แม้ Word Embedding จะประสบความสำเร็จในการแปลง "คำ" ให้เป็นตัวเลขที่มีมิติเชิงความหมาย แต่ภาษาธรรมชาติคือการร้อยเรียงกันของลำดับเหตุการณ์ โครงข่ายประสาทเทียมแบบดั้งเดิม (Feed-Forward Neural Networks หรือ ANN) ขาดคุณสมบัติที่สำคัญที่สุดประการหนึ่งในการประมวลผลภาษา นั่นคือกลไก "ความจำ" (Memory) โมเดลเหล่านี้รับข้อมูลเข้าและส่งออกไปในทิศทางเดียว โดยไม่สนใจลำดับก่อนหลัง (Order of occurrence) ทำให้มันไม่สามารถเข้าใจโครงสร้างของประโยคที่ซับซ้อนได้

แนวคิดเพื่อแก้ปัญหานี้ถูกนำเสนอผ่านสถาปัตยกรรม Recurrent Neural Networks (RNN) ซึ่งอนุญาตให้มีกลไกป้อนกลับ (Feedback Loop) ภายในเซลล์ประสาท การตัดสินใจหรือผลผลิต (Output) ในขั้นเวลาปัจจุบัน (t) จะถูกนำไปรวมกับข้อมูลใหม่ที่จะเข้าสู่ระบบในขั้นเวลาถัดไป ($t + 1$) ในลักษณะของสถานะซ่อนเร้น (Hidden State) ทำให้เกิดการส่งผ่านบริบท (Context Flow) อย่างไรก็ตาม RNN แบบมาตรฐานมีจุดอ่อนที่รุนแรงในเชิงวิศวกรรมคณิตศาสตร์ คือปัญหาการลดทอนของความชัน (Vanishing Gradient Problem) เมื่อลำดับข้อมูลมีความยาวเพิ่มขึ้น การคูณย้อนกลับ (Backpropagation) ผ่านลำดับชั้นเวลาที่ยาวไกลจะทำให้ค่าความชันลดระดับลงจนเข้าใกล้ศูนย์ ส่งผลให้โมเดลเกิดสภาวะ "ความจำสั้น" ไม่สามารถรับรู้บริบทของต้นประโยคเมื่ออ่านมาถึงท้ายประโยคได้

เพื่อทลายข้อจำกัดดังกล่าว สถาปัตยกรรม Long Short-Term Memory (LSTM) จึงถูกประดิษฐ์ขึ้น ความแตกต่างที่ทำให้ LSTM เหนือกว่า RNN คือการนำเสนอแนวคิดของ "สถานะเซลล์" (Cell State) ซึ่งทำหน้าที่เหมือนสายพานลำเลียง (Conveyor Belt) ที่ไหลผ่านตลอดทั้งเครือข่าย โดยมีปฏิสัมพันธ์เชิงเส้นเพียงเล็กน้อย ทำให้ข้อมูลความทรงจำระยะยาว (Long-Term Dependencies) สามารถส่งผ่านไปได้โดยไม่เกิดการสูญหาย กลไกในการควบคุมเส้นทางของข้อมูลเข้าและออกสถานะเซลล์ถูกควบคุมอย่างเข้มงวดด้วย "ประตู" (Gates) จำนวน 3 ชุด ซึ่งประกอบด้วยสถาปัตยกรรมย่อยดังนี้ :

1.3.1. ประตูการลืม (Forget Gate): ทำหน้าที่พิจารณาสถานะซ่อนเร้นจากอดีต (h_{t-1}) ร่วมกับข้อมูลชุดใหม่ (x_t) นำไปผ่านฟังก์ชันกระตุ้น Sigmoid เพื่อประเมินค่าความจำเป็น ค่าส่งออกจะเป็นสเกลาร์ระหว่าง 0 ถึง 1 โดย 0 หมายถึง "ลบความจำนี้ทิ้งทั้งหมด" และ 1 หมายถึง "เก็บความจำนี้ไว้ทั้งหมด" ขั้นตอนนี้มีความสำคัญในการกำจัดบริบทที่ล้าสมัยหรือไม่เกี่ยวข้องแล้วออกไปจากเส้นทาง

1.3.2. ประตูการนำเข้า (Input Gate): ทำหน้าที่ 2 ส่วนควบคู่กัน ส่วนแรกใช้ฟังก์ชัน Sigmoid เพื่อตัดสินใจว่าข้อมูลใหม่ส่วนใดควรค่าแก่การจัดเก็บ (อัปเดตสถานะเซลล์) ส่วนที่สองใช้ฟังก์ชัน Tanh เพื่อสร้างเวกเตอร์สถานะใหม่ (New Candidate Values) ข้อมูลทั้งสองส่วนจะถูกคูณแบบจุดต่อจุด (Pointwise Multiplication) และนำไปบวกรวมกับสถานะเซลล์ที่ผ่านการลืมมาแล้ว เพื่อหล่อหลอมเป็นความทรงจำใหม่ที่ทันสมัย

1.3.3. ประตูการส่งออก (Output Gate): ทำหน้าที่กลั่นกรองสถานะเซลล์ปัจจุบัน (C_t) เพื่อสร้างเป็นสถานะซ่อนเร้น (Hidden State, h_t) สำหรับขั้นเวลาปัจจุบัน โดยใช้ฟังก์ชัน Sigmoid ในการประเมินว่าควรดึงคุณลักษณะใดจากเซลล์ออกมา และใช้ฟังก์ชัน Tanh สเกลค่าสถานะเซลล์ก่อนที่จะนำมาผสมรวมกัน เพื่อส่งต่อไปเป็นข้อมูลป้อนเข้าในขั้นเวลาถัดไป รวมถึงส่งต่อให้ชั้นโครงข่ายอื่นๆ ในสถาปัตยกรรมรวม

1.4. โมเดลการอ่านข้อมูลแบบสองทิศทาง (Bidirectional LSTM)

แม้ LSTM แบบทิศทางเดียว (Unidirectional LSTM) จะสามารถจดจำบริบทในอดีต (Past Context) ได้อย่างยอดเยี่ยม แต่ตรรกะในการทำความเข้าใจภาษาไม่ได้อ้างอิงถึงอดีตเสมอไป ในกระบวนการอ่านของมนุษย์ สมองมักจะพิจารณาทั้งคำที่ผ่านมาแล้วและคำที่กำลังจะเกิดขึ้นเพื่อสร้างมโนทัศน์เกี่ยวกับความหมาย ตัวอย่างเช่น หากพบคำว่า "ดี" ระบบไม่สามารถระบุอารมณ์ได้จนกว่าจะอ่านพบคำถัดไป เช่น "ดีเด็ก" (แง่ลบ) หรือ "ดีตัว" (คำกริยาทั่วไป)

สถาปัตยกรรม Bidirectional LSTM (Bi-LSTM) ได้ถูกออกแบบเพื่อจำลองความสามารถนี้ โดยการทำงานประสานกันของชั้น LSTM จำนวน 2 ชุด (Forward LSTM และ Backward LSTM) ขนานกันไป ชุดหนึ่งประมวลผลประโยคจากซ้ายไปขวา (ต้นไปจบ) ในขณะที่อีกชุดหนึ่งประมวลผลประโยคจากขวามาซ้าย (จบมาต้น) ทวนเข็มทิศทางเดิม ท้ายที่สุด เวกเตอร์สถานะซ่อนเร้นจากทั้งสองทิศทาง (Forward Hidden State h_t และ Backward Hidden State $\overleftarrow{h}_t$) จะถูกนำมาผสมรวมกัน (Concatenation) ตามสมการ :

$$h_t = \overrightarrow{LSTM}(x_t)$$

$$\overleftarrow{h}_t = \overleftarrow{LSTM}(x_t)$$

$$h_t = [h_t \oplus \overleftarrow{h}_t]$$

การผสมผสานองค์ความรู้ทั้งจากอดีตและอนาคตทำให้โมเดลได้เรียนรู้บริบทแวดล้อมโดยสมบูรณ์รอบด้าน ส่งผลให้ค่าความแม่นยำในการแยกแยะบริบทที่ซับซ้อนและการวิเคราะห์อารมณ์ความรู้สึกเชิงลึกเพิ่มขึ้นอย่างมีนัยสำคัญเหนือสถาปัตยกรรมแบบทิศทางเดียวอย่างเห็นได้ชัด

2. วิศวกรรมข้อมูลและการเตรียมความพร้อม (Data Engineering Pipeline)

ความสำเร็จของแบบจำลองปัญญาประดิษฐ์ขึ้นอยู่กับคุณภาพของข้อมูลอย่างหลีกเลี่ยงไม่ได้ (Garbage In, Garbage Out) กระบวนการทางวิศวกรรมข้อมูลสำหรับการประมวลผลภาษาธรรมชาติ (Text Data Engineering) มีความซับซ้อนสูงและประกอบด้วยขั้นตอนเชิงปฏิบัติการที่เป็นรากฐานสำคัญหลายประการ ดังนี้:

2.1. การทำความสะอาดข้อมูล (Data Cleaning) และสมดุลของข้อมูล (Class Distribution)

ข้อมูลดิบที่ถูกรวบรวมมักจะมีเสียงรบกวน (Noise) การทำความสะอาดเริ่มต้นจากการสแกนเอกสารข้อความเพื่อคัดแยกและลบบรรทัดที่มีลักษณะบกพร่องออกไป เช่น บรรทัดที่ว่างเปล่า บรรทัดที่มีการระบุป้ายกำกับ (Label) ไม่ครบถ้วน หรือมีการพิมพ์ผิดเพี้ยนไปจากมาตรฐาน รวมถึงการคัดกรองประโยคที่ซ้ำซ้อน (Duplicated Data) ซึ่งมักพบได้บ่อยในชุดข้อมูลที่ดึงมาจากโซเชียลมีเดีย การปล่อยให้ข้อมูลซ้ำซ้อนจะสร้างอคติ (Bias) ทำให้โมเดลจดจำแพทเทิร์นที่เฉพาะเจาะจงมากเกินไปจนเกิดปัญหา Overfitting

นอกจากนั้น จะต้องมีการวิเคราะห์การกระจายตัวของคลาสข้อมูล (Class Distribution) ซึ่งในบริบทของการวิเคราะห์อารมณ์ความรู้สึก (Sentiment Analysis) มักจะประกอบด้วยคลาสเชิงบวก (Positive), คลาสเชิงลบ (Negative) และคลาสทั่วไป (Neutral) สัดส่วนของประโยคในแต่ละหมวดหมู่ควรมีความใกล้เคียงหรือสมดุลกัน (Balanced Dataset) เพื่อป้องกันไม่ให้โครงข่ายประสาทเทียมเอนเอียงการทำนายไปที่กลุ่มข้อมูลส่วนใหญ่เพียงอย่างเดียว หากพบว่าสัดส่วนมีความแตกต่างกันเพียงเล็กน้อยก็สามารถดำเนินการกระบวนการต่อได้ แต่หากมีความเหลื่อมล้ำสูง จำเป็นต้องใช้เทคนิคเพิ่มเติม เช่น Undersampling หรือ Oversampling เข้ามาช่วยปรับคุณภาพ

2.2. การจัดรูปแบบโครงสร้างข้อมูล (Formatting Dataset) และการตัดคำ (Tokenization)

การจัดการโครงสร้างไฟล์ชุดข้อมูลมักกำหนดให้มีรูปแบบที่อ่านค่าได้ง่าย เช่น [ข้อความ] <ตัวค้น>[ป้ายกำกับ] ระบบคอมพิวเตอร์จะใช้ฟังก์ชันการตัดสายอักขระ (String Split Method) โดยระบุตำแหน่งตัวค้นเพื่อแยกชิ้นส่วนของประโยคออกจากคำตอบ ข้อความที่เป็นบริบทจะถูกส่งผ่านฟังก์ชันสำหรับลบช่องว่างส่วนเกินที่บริเวณหัวและท้าย (Strip Whitespaces) ก่อนที่จะส่งเข้าไปยังไลบรารีตัดคำ (เช่น

Deepcut) เมื่อการหั่นประโยคออกเป็นหน่วยย่อย (Tokens) เสร็จสิ้น ข้อมูลจะถูกบันทึกแยกออกเป็น 2 ส่วน สำคัญ ได้แก่ ส่วนที่เก็บเฉพาะรายการคำศัพท์ล้วน ๆ ไม่มีป้ายกำกับเจือปนเพื่อใช้สำหรับการฝึกสอนโมเดล แปลภาษา (Corpus) และส่วนชุดข้อมูลรูปแบบสมบูรณ์เพื่อการฝึกสอนโมเดลโครงข่ายประสาทเทียมสำหรับการจำแนกอารมณ์ในท้ายที่สุด

2.3. การฝึกสอนและบันทึกโมเดล Word2Vec

การจำลององค์ความรู้ความหมายของคำผ่านโมเดล Word2Vec จำเป็นต้องมีการกำหนด พารามิเตอร์เชิงวิศวกรรมที่เหมาะสมเพื่อสะท้อนความซับซ้อนของภาษา :

2.3.1. ขนาดมิติเวกเตอร์ (vector_size = 100) กำหนดให้กระบวนการแปลงรหัสคำให้ กลายเป็นคุณลักษณะต่อเนื่องจำนวน 100 แกน ซึ่งเป็นค่ามาตรฐานที่สร้างความสมดุลระหว่างความ ละเอียดลออทางความหมายและการคำนวณทางคณิตศาสตร์

2.3.2. ขนาดของหน้าต่างบริบท (window = 5) เป็นการกำหนดขอบเขตที่สนใจในการ พิจารณาบริบท โดยระบุให้โมเดลมองขยายออกไปยังคำที่อยู่ก่อนหน้า 5 ตำแหน่ง และคำที่อยู่ตามหลัง 5 ตำแหน่ง ซึ่งครอบคลุมระยะโครงสร้างประโยคมาตรฐานของภาษาไทยได้อย่างมีประสิทธิภาพ

2.3.3. ความถี่ขั้นต่ำ (min_count = 1) บังคับให้ระบบต้องเรียนรู้และแปลงเวกเตอร์สำหรับ คำศัพท์ทุกคำแม้จะปรากฏเพียงแค่หนึ่งครั้งในฐานข้อมูล (Frequency = 1) เพื่อไม่ให้เกิดการละทิ้งคำวิเศษณ์ แสดงอารมณ์หรือสแลงที่มีความหมายเฉพาะตัวสูง

2.3.4. สถาปัตยกรรมการเรียนรู้ (sg = 1) กำหนดให้เรียกใช้งานอัลกอริทึม Skip-Gram แทนที่ Continuous Bag-of-Words (CBOW) โดยมีจุดประสงค์หลักเพื่อเพิ่มประสิทธิภาพในการสกัดบริบท และเสริมความสามารถในการเรียนรู้คำที่หายากให้มีความลุ่มลึกมากยิ่งขึ้น

2.3.5. การกระจายภาระการคำนวณ (workers) ใช้งานหน่วยประมวลผล (CPU) ทรัพยากร ที่มีอยู่ทุกแกนหลักเพื่อประมวลผลแบบขนาน (Multiprocessing) ทำให้กระบวนการแปลงและเรียนรู้ชุด คำศัพท์สำเร็จได้อย่างรวดเร็ว

2.4. การกำหนดรหัสผ่านอาร์เรย์ (Encoding) และการจัดรูปแบบความยาวประโยค (Sequence Padding)

ก่อนที่จะป้อนข้อมูลเข้าสู่โครงข่าย LSTM ชุดข้อมูลจะต้องถูกแปลงสภาพจากข้อความให้ กลายเป็นเมทริกซ์และอาร์เรย์ การแปลงค่าเริ่มต้นด้วยการสร้างพจนานุกรมดัชนีคำศัพท์ (Word Index) ที่ กำหนดบัตรคิวตัวเลขให้กับคำศัพท์ทุกคำ (เช่น 'อาหาร' = 15, 'อร่อย' = 8) โดยทำการสงวนรหัสหมายเลข 0 ไว้สำหรับวัตถุประสงค์พิเศษ

ประเด็นทางวิศวกรรมคือประโยคที่มนุษย์สร้างสรรค์ขึ้นมีความยาวที่ไม่แน่นอน แต่การดำเนินการคุณเมทริกซ์ในระดับโครงข่ายประสาทเทียมต้องการรูปแบบมิติที่ตายตัว (Fixed Dimensions) อุปสรรคนี้แก้ไขได้โดยเทคนิคการเติมและตัดข้อมูล (Sequence Padding) :

2.4.1. การกำหนดความยาวคงที่ (maxlen = 50): นิยามให้เมทริกซ์ข้อมูลเข้ามีขนาดยาวสูงสุดที่ 50 ตำแหน่งต่อประโยค

2.4.2. การเติมเต็ม (Zero-Padding): สำหรับประโยคที่มีความยาวต่ำกว่าเกณฑ์ (ตัวอย่างเช่น มีคำศัพท์ 20 คำ) ระบบจะนำรหัสหมายเลข 0 เข้าไปอุดช่องว่างที่เหลือจนครบ 50 เพื่อรักษาขนาดของอาร์เรย์ให้เท่ากัน

2.4.3. การตัดทอน (Truncation): สำหรับประโยคที่ยาวเกินขีดจำกัด (เช่น ประโยควิจารณ์ขนาดยาว) ระบบจะตัดคำศัพท์ในส่วนเกินทิ้งไป

ในส่วนของป้ายกำกับอารมณ์เป้าหมาย (Target Labels) ซึ่งถูกแปลงรูปเป็นรหัสเชิงกลุ่ม (เช่น 0 = เชิงบวก, 1 = เชิงลบ, 2 = ทัวไป) ระบบจะใช้กระบวนการ One-Hot Encoding (to_categorical) ในการเปลี่ยนค่าตัวเลขเดี่ยวให้เป็นอาร์เรย์ของการกระจายตัวความน่าจะเป็นเชิงพื้นที่ (Probability Distribution) เช่น เลข 1 ถูกสลายเป็น `` เพื่อความสอดคล้องกับค่าผลลัพธ์ที่เกิดจากฟังก์ชันสมการชั้นสุดท้ายของปัญญาประดิษฐ์

3. สถาปัตยกรรมโครงข่ายประสาทเทียมระดับสูงเพื่อการจำแนกข้อความ (Model Architecture & Training)

การรังสรรค์สมองกลเพื่อการวิเคราะห์ข้อความ (Text Classification) อาศัยโครงสร้างโมเดลแบบไล่ลำดับชั้น (Sequential Model) ซึ่งทำงานโดยส่งผ่านร่องรอยการประมวลผลไปข้างหน้าทีละระดับ โครงสร้างสถาปัตยกรรม (Architecture) ของโมเดลมีการกำหนดรายละเอียดในแต่ละชั้นปฏิบัติการอย่างลึกซึ้งดังนี้ :

3.1. ชั้นเวกเตอร์การฝังคำ (Embedding Layer)

ชั้นแรกสุดเปรียบเสมือนศูนย์บัญชาการแปลภาษา ทำหน้าที่แปลงรหัสบิตตัวเลขให้เป็นปริภูมิเวกเตอร์ความหมาย 100 มิติ โดยใช้เทคนิคการถ่ายโอนการเรียนรู้ (Transfer Learning) จากการนำเมทริกซ์น้ำหนักของโมเดล Word2Vec ที่ได้สร้างเตรียมไว้มาฝังเป็นค่าเริ่มต้น พารามิเตอร์ที่สำคัญอย่างยิ่งยวดในชั้นนี้คือ trainable = True ซึ่งเป็นคำสั่งปลดล็อกให้โครงข่ายประสาทเทียมมีอิสระในการปรับจูน (Fine-tuning) เวกเตอร์คำศัพท์ใหม่ในระหว่างกระบวนการเรียนรู้ การเปิดโอกาสให้โมเดลปรับเปลี่ยนน้ำหนักเหล่านี้ช่วยให้ตัวแบบไม่ยึดติดกับความสัมพันธ์ของศัพท์เชิงพจนานุกรมมากเกินไป แต่เปิดรับการปรับทิศทางของคำศัพท์ให้สอดคล้องกับ "บริบทอารมณ์" (Sentiment Context) มากยิ่งขึ้น

3.2. ชั้นหน่วยความจำระยะยาวแบบสองทิศทาง (Bidirectional LSTM Layer)

ชั้นประมวลผลตรรกะหัวใจหลักของระบบ โมเดลใช้โครงข่าย LSTM สองชุดอ่านทวนกันไปมา (ทิศทางไปข้างหน้าและทิศทางย้อนกลับ) ควบคู่กับการประมวลผลเชิงลำดับ โดยระบุโหนดเซลล์ประสาทสำหรับการพิจารณาถึง 128 หน่วย (Units) อีกทั้งมีการตั้งค่าคุณลักษณะ `return_sequences = False` ซึ่งสั่งการให้โมเดลไม่จำเป็นต้องให้ผลลัพธ์ระหว่างทางในแต่ละช่วงก้าวเวลา (Time Steps) แต่ทำการวิเคราะห์รวบยอดประโยคทั้งหมดตั้งแต่ต้นจนจบก่อน แล้วจึงส่งผ่านเวกเตอร์สถานะซ่อน (Final Hidden State) ไปยังโครงข่ายลำดับถัดไป ซึ่งเป็นแนวทางปฏิบัติมาตรฐานสำหรับงานจำแนกประเภทเอกสารหรือประโยคในภาพรวม

3.3. ชั้นป้องกันการเรียนรู้เกินพอดี (Dropout Layer)

การใช้โครงสร้างปัญญาประดิษฐ์ที่ซับซ้อนย่อมแลกมาด้วยความเสี่ยงในการเกิดภาวะท่องจำคำตอบ (Overfitting) ซึ่งโมเดลจะแม่นยำสูงเมื่อพบโจทย์เดิมแต่กลับสอบตกเมื่อเจอข้อมูลที่ไม่คุ้นเคย การบรรเทาปัญหาถูกกระทำผ่านเลเยอร์ Dropout โดยมีการตั้งค่าไว้ที่ระดับ 0.5 (ร้อยละ 50) ส่งผลให้ในการวนรอบประมวลผลแต่ละรอบ เซลล์ประสาทครึ่งหนึ่งจะถูกสุ่มสั่งปิดการทำงาน (Random Deactivation) โดยอัตโนมัติ กระบวนการนี้บีบบังคับให้เซลล์ที่ยังคงทำงานอยู่ไม่สามารถพึ่งพากลไกการจดจำจากโหนดใดโหนดหนึ่งได้ และต้องเรียนรู้คุณลักษณะทั่วไปของข้อมูล (Generalization) อย่างแข็งแกร่งยิ่งขึ้น

3.4. ชั้นคัดแยกหมวดหมู่หนาแน่น (Dense Layer & Softmax Activation)

การตัดสินใจขั้นสุดท้ายถูกดำเนินการโดยเครือข่ายเชื่อมต่อสมบูรณ์ (Fully Connected Layer หรือ Dense Layer) ซึ่งมีจำนวนโหนดทางออก (Output Units) เท่ากับ 3 ช่องทาง เพื่อให้สอดคล้องกับป้ายกำกับอารมณ์ทั้ง 3 กลุ่ม (เชิงบวก เชิงลบ เป็นกลาง) โดยผนวกฟังก์ชันกระตุ้นสมองแบบ Softmax ซึ่งมีคุณสมบัติทางคณิตศาสตร์ในการแปลงค่าดิบของน้ำหนักให้อยู่ในรูปแบบของช่วงระดับเปอร์เซ็นต์ หรือการกระจายความน่าจะเป็น (Probability Distribution) ซึ่งผลรวมเปอร์เซ็นต์ของทุกโหนดจะต้องมีค่าเป็น 1 หรือ 100% เสมอ

3.5. การคอมไพล์และการควบคุมวงจรการเรียนรู้ (Compilation & Callbacks)

ก่อนจะเริ่มฝึกสมองกล โมเดลถูกผสมผสานรวม (Compiled) โดยใช้ 옵ติไมเซอร์แบบ Adaptive Moment Estimation (Adam) สำหรับการควบคุมความชัน และเลือกใช้ฟังก์ชันประเมินความคลาดเคลื่อน (Loss Function) แบบ Categorical Crossentropy ซึ่งเหมาะสมอย่างยิ่งกับผลลัพธ์แบบหลายหมวดหมู่

ในกระบวนการฝึกฝนแบบบูรณาการ (Training Process) มีการตั้งรอบการอ่านชุดเอกสารทั้งหมดไว้ 30 รอบประวัติศาสตร์ (Epochs) พร้อมแบ่งหน่วยข้อมูลย่อยเข้ากระบวนการครั้งละ 32 บรรทัด ประโยค (Batch Size) ตลอดจนการกันชุดข้อมูลออกมาจำนวน 20% (Validation Split = 0.2) เพื่อทำหน้าที่เป็นกระดาษข้อสอบลับในการประเมินความสามารถที่แท้จริง ความน่าสนใจคือระบบไม่ได้ถูกปล่อยให้ทำงาน

โดยอิสระไร้ทิศทาง แต่นำผู้ช่วยควบคุมคุณภาพเครือข่าย (Callbacks) เข้ามาร่วมพิจารณาอย่างใกล้ชิดถึง 2 กลไกด้วยกัน ได้แก่ :

3.5.1. จุดสกัดกันเพื่อกันความสูญเสียเปล่า (EarlyStopping): ระบบทำการประเมินค่าความแม่นยำในกลุ่มข้อสอบลับ หากเส้นกราฟแสดงให้เห็นว่าไม่มีการพัฒนาหรือค่าความผิดพลาด (Validation Loss) ไม่ลดระดับลงเลยอย่างต่อเนื่องถึง 3 สมัยประวัติศาสตร์ (patience = 3) ระบบจะระงับการเทรน กระทั่งกัน เพื่อรักษาเสถียรภาพและไม่เสียเวลาโดยเปล่าประโยชน์

3.5.2. ผู้พิทักษ์น้ำหนักที่ประเสริฐสุด (ModelCheckpoint): ตลอดช่วงวงจรการฝึกสอน ทุกครั้งที่ปัญหาประติษฐ์ทำลายสถิติคะแนนความแม่นยำได้สูงสุด (Best Validation Score) เครื่องมือตัวนี้จะลอกเลียนสำเนาน้ำหนัก ณ โมเมนต์เหล่านั้นแล้วทำการบันทึกข้อมูลย่อยเก็บไว้ในเครื่องจักรที่รับประกันว่าปัญหาประติษฐ์ในสภาวะที่ดีที่สุดจะไม่สูญหายแม้รอบหลังสุดจะสอบตกก็ตาม

4. การวัดผลและประเมินประสิทธิภาพการวิเคราะห์อารมณ์ (Evaluation Metrics for Sentiment Analysis)

การประเมินค่าผลลัพธ์ในงานประเภทที่มีหมวดหมู่จำนวนมากและไม่มีความต่อเนื่องของทิศทาง (Multi-class Classification) ถือเป็นจุดวิกฤติที่หลายคนมองข้าม การอาศัยพึ่งพาตัวชี้วัดความแม่นยำในภาพรวม (Accuracy) เป็นสิ่งที่ไม่แนะนำ เนื่องจาก Accuracy ไม่สามารถสะท้อนความสามารถเชิงลึกของโมเดลเมื่อต้องรับมือกับความไม่สมดุลของข้อมูล (Imbalanced Datasets) ได้ ตัวอย่างเช่น หากมีข้อมูลประโยคเชิงลบเพียงร้อยละ 5 และระบบสุ่มทำนายเป็นเชิงบวกทั้งหมด ก็ยังคงได้ค่า Accuracy มหาศาลถึง 95% ทั้งที่ในบริบทความเป็นจริง โมเดลนี้ไร้ประโยชน์และไม่สามารถบรรลุวัตถุประสงค์ในการระบุข้อพิพาทของลูกค้าได้เลย

เพื่อป้องกันหลุมพรางทางสถิติดังกล่าว นักวิจัยระบบเรียนรู้ปัญหาประติษฐ์จึงหันไปใช้เครื่องมือชี้วัดที่มีความจำเพาะเจาะจงมากขึ้น โดยอ้างอิงและพัฒนาต่อยอดมาจากโครงข่ายตารางความสับสน (Confusion Matrix) ซึ่งแบ่งผลลัพธ์ย่อยออกมาตรวัดที่สำคัญได้ 3 ประการหลัก :

มาตรวัดประเมินผล	ปรัชญาและสมการประเมิน	ความสำคัญและกรณีการประยุกต์ใช้จริง
ความแม่นยำของผลทำนาย (Precision)	สูตร: $\text{Precision} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalsePositives}}$ สะท้อนอัตราส่วนความถูกต้องเมื่อพิจารณาเฉพาะกลุ่มก่อนที่ตัวแบบระบุว่า "จริง"	เป็นมาตรวัดที่ตอบคำถามว่า "หากโมเดลแจ้งเตือนว่าเป็นประโยคเชิงลบ จะเชื่อถือได้กี่เปอร์เซ็นต์?" เหมาะอย่างยิ่งสำหรับสภาพแวดล้อมที่ผลบวกหลง (False Positive) ส่งผลร้ายต่อระบบ หรือมีต้นทุนการบริหารจัดการสูง
ความครอบคลุม (Recall / Sensitivity)	สูตร: $\text{Recall} = \frac{\text{TruePositives}}{\text{TruePositives} + \text{FalseNegatives}}$ สะท้อนสัดส่วนที่ปัญหาประดิษฐ์สามารถค้นพบตัวอย่างที่เป็น "จริง" จากฐานข้อมูลที่ต้องจับตามองทั้งหมด	เป็นมาตรวัดที่ตอบคำถามว่า "ในบรรดาประโยคเชิงลบที่มีอยู่จริง โมเดลจับได้ครอบคลุมกี่เปอร์เซ็นต์?" เหมาะกับการแก้ปัญหาที่ต้องการหลีกเลี่ยงผลลบหลง (False Negative) เพราะการปล่อยปะละเลยประโยคอันตรายอาจบานปลายจนคุมไม่ได้
คะแนนเอฟวัน (F1-Score)	สูตร: $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ เป็นค่าเฉลี่ยฮาร์โมนิก (Harmonic Mean) ซึ่งสร้างสมดุลระหว่างสองมาตรวัด	ในเชิงปฏิบัติ Precision และ Recall เป็นไม้เบื่อไม้เม้าที่สวนทางกัน (Trade-off) การผลักดัน F1-Score ถือเป็นการรักษาสมดุลเชิงผลประโยชน์โดยรวม ทำให้ผู้พัฒนาสามารถรับรู้เสถียรภาพของอัลกอริทึมได้เป็นอย่างดีที่สุด โดยเฉพาะในกรณีวิเคราะห์คำศัพท์หลายกลุ่มซ้ำ

5. แนวคิดขั้นสูงและการแก้ปัญหาความซับซ้อนเชิงเทคนิค (Advanced Concepts & Troubleshooting)

เพื่อให้ครอบคลุมขอบเขตการศึกษาระดับผู้เชี่ยวชาญ การตระหนักรู้ถึงข้อจำกัดตลอดจนปัญหาทางเทคนิคของกระบวนการทศน์ NLP จะช่วยยกระดับความสามารถในการพัฒนาระบบ การรับมือความซับซ้อนในวิศวกรรมภาษาไทยมีประเด็นที่น่าสนใจดังนี้:

5.1. ปัญหาของคำศัพท์ปริศนานอกพจนานุกรม (Out-Of-Vocabulary: OOV)

จุดอ่อนแฝงประการสำคัญของการใช้แบบจำลอง Word-level Embedding เช่น Word2Vec คือลักษณะการจำกัดการเรียนรู้ไว้เพียงฐานพจนานุกรมที่มีอยู่เท่านั้น เมื่อโมเดลเหล่านี้ถูกนำไปใช้งานร่วมกับข้อมูลจริงในโซเชียลมีเดียที่มักเต็มไปด้วยปัญหาการสะกดผิด การเล่นคำ หรือวิวัฒนาการทางภาษา ศัพท์ใหม่เหล่านี้จะไม่สามารถถูกประเมินค่าเวกเตอร์ได้ (กลายเป็นตัวแปรที่ไม่รู้จัก) และสูญหายจากเมทริกซ์การตีความทันที ทำให้ประสิทธิภาพการรับรู้บริบทประโยคระดับลงอย่างมีนัยสำคัญ

เพื่อชดเชยรอยร้าวนี้ นักวิจัยด้านปัญญาประดิษฐ์เสนอการแก้ปัญหาผ่านสถาปัตยกรรมกลุ่ม Sub-word Information อย่าง FastText โมเดลนี้ถูกพัฒนาครอบทับรากฐาน Word2Vec เดิม แต่ปรับมุมมองให้หันไปพิจารณาการแยกโครงสร้างคำเชิงสัณฐานวิทยา (Morphological Structure) ผ่านลำดับตัวอักษรแบบเอ็นแกรม (n-grams) ควบคู่ไปด้วย ส่งผลให้ถึงแม้ระบบจะไม่เคยพบศัพท์ "อร่อยยย" แบบเต็มคำมาก่อน แต่ด้วยโครงสร้างส่วนย่อยที่เป็น n-grams ของคำว่า "อร่อย" โมเดล FastText ก็ยังสามารถอนุมานตำแหน่งเวกเตอร์ความหมายที่ใกล้เคียงกันได้อย่างรวดเร็ว อย่างไรก็ตาม การทำงานพิจารณาอักขระส่วนย่อยเหล่านี้ย่อมต้องแลกมาด้วยระยะเวลาและทรัพยากรการเทรนที่เพิ่มขึ้นอย่างมหาศาลเมื่อเปรียบเทียบกับ CBOW หรือ Skip-Gram ในตระกูล Word2Vec ดั้งเดิม

5.2. ภาวะความท้าทายเชิงไวยากรณ์ (Syntactic and Semantic Nuance)

เทคโนโลยี NLP ยังคงเผชิญอุปสรรคกับความละเอียดอ่อนทางภาษา (Nuance) เช่น การประชดประชัน (Sarcasm) ประโยคเสียดสี การแสดงออกเชิงบริบทแฝง หรือกลไกการปฏิเสธแบบซ้อนรูป (Negation) ซึ่งแม้แต่การใช้ Bidirectional LSTM ก็ยังอาจนำเสนอคำทำนายที่บิดเบี้ยวได้ หากขนาดข้อมูลฝึกสอนไม่มีความหลากหลายเชิงอารมณ์มากพอ เพื่อบรรเทาปัญหานี้ งานวิจัยเชิงลึกพยายามสอดแทรกคุณลักษณะฟิเจอร์อื่นๆ เพิ่มเติมเข้าไปในโมเดล เช่น โครงสร้างรูปแบบทางไวยากรณ์ (Part-of-Speech: POS tag) เพื่อสะท้อนความเชื่อมโยงของชนิดคำ หรือการสอดแทรกอรรถศาสตร์เชิงอารมณ์จากโครงข่ายนอกระบบ (Sentiment Features) เพื่อถ่วงน้ำหนักคลาสที่เป็นปัญหาให้ดีขึ้น

6. การประยุกต์ใช้ในสถานการณ์จริงและบทสรุป (Practical Application & Conclusion)

6.1. กลไกอนุมานประมวลผลและการใช้งานจริง (Inference & Deployment Pipeline)

เมื่อผ่านการฝึกสอนแบบจำลองจนมีความสมบูรณ์ทางสถาปัตยกรรม การนำเอาระบบสมองกลอัจฉริยะ (Deployment) ไปใช้ตอบสนองโจทย์ปัญหาทำงาน จำเป็นต้องมีกลไกอนุมาน (Inference Pipeline) ที่ประกอบไปด้วยการประสานกระบวนการหลายขั้นตอนเข้าด้วยกันอย่างราบรื่น :

6.1.1. การปลูกชีพื้นฐานข้อมูลและน้ำหนักรวม: เริ่มต้นด้วยการดึงระบบตารางเทียบรหัสประจำคำศัพท์ (Word Index Dictionary) เพื่อใช้ในฐานะวุ่นแปลภาษา และดำเนินการโหลดน้ำหนักโครงสร้างปัญญาประดิษฐ์ LSTM จากไฟล์เซฟ .h5 กลับคืนสู่หน่วยความจำหลักพร้อมคำสั่ง

6.1.2. ดักจับสายอักขระสื่อสาร: ทำการรับฟังตัวแปรข้อความดิบจากระบบปฏิสัมพันธ์ระหว่างมนุษย์ (User Input) ในลักษณะลูปประมวลผลต่อเนื่อง (Continuous Polling or While Loop)

6.1.3. การปรับเปลี่ยนและกลั่นกรองบริบท (Pre-processing Phase): ป้อนข้อความใหม่เข้าไปสู่การวิเคราะห์ตัวแปรระดับคำศัพท์ (Tokenization) เพื่อทำลายกำแพงอักขระ ผ่านการสับคั่นและปรับรหัสตามฐานข้อมูลดัชนีนาเรีย และบังคับยัด-หุดอาร์เรย์ให้อยู่ภายใต้กรอบข้อจำกัด 50 โทเคนเปื้อ (Sequence Padding) ด้วยชุดคำสั่ง pad_sequences เพื่อให้โครงข่ายเซลล์ประสาทเทียมสามารถรับข้อมูลเข้าได้อย่างไร้ปัญหา

6.1.4. การทำนายทิศทางความเชื่อมั่น (Prediction and Confidence Scoring): ดำเนินการโยนชุดข้อมูล 1 มิติ เข้าปะทะกับพารามิเตอร์ของเลเยอร์ LSTM เมื่อระบบทำการกรองผ่านความทรงจำ 2 ทิศทาง และรีดข้อมูลผ่านชั้น Softmax ผลผลิตจะถูกคืนค่ากลับมาในมาตรวัดความน่าจะเป็นระดับทศนิยม 3 ช่องทางชั่วอารมณ์

6.1.5. การแปลผลรหัสกลับสู่มนุษย์ (Decoding & Output): นำฟังก์ชันอาร์กิวเมนต์ทางสถิติ (เช่น np.argmax) ไปจับตำแหน่งตัวเลขคะแนนที่สูงโด่งเหนือตัวเลือกอื่น จากนั้นไปหาเบงข้อมูลรหัสเป้าหมาย (id2label) เช่น เลข 0 แปลงร่างเป็นหมวดหมู่เชิงบวก ท้ายที่สุด ตัวระบบจะทำการพ่นหรือแสดงผลลัพธ์ว่าข้อความสะท้อนอารมณ์ใดออกมาควบคู่กับตัวเลขเปอร์เซ็นต์ความมั่นใจ เพื่อให้มนุษย์นำตรรกะข้อนี้ไปปฏิบัติงานต่อ

6.2. บทสรุปและทิศทางในอนาคต (Conclusion & Future Outlook)

การศึกษาวิทยาการทางด้านการประมวลผลภาษาธรรมชาติในชุดองค์ความรู้นี้ ถือเป็นการสกัดแก่นแท้เชิงพฤติกรรมระหว่างการจัดของปัญญาประดิษฐ์ (AI) กับการเรียนรู้ของเครื่องจักรวิเคราะห์เชิงลึก (Deep Learning) นับตั้งแต่การทำลายข้อจำกัดเชิงโครงสร้างอักขระผ่านกระบวนการ Tokenization , การผลึกโฉมองค์ความรู้จากปริภูมิความหมายเชิงสัญลักษณ์ให้เป็นโลกที่คำนวณได้ทางเวกเตอร์ (Word2Vec Skip-Gram Architecture) , กระทั่งสามารถประกอบสร้างโครงข่ายระบบความทรงจำระยะยาวที่มีการตีความทั้งไปและกลับได้สมบูรณ์ (Bidirectional LSTM) สิ่งเหล่านี้คือทักษะรากฐานแห่งการสร้างโครงข่ายประสาทที่มีความตระหนักรู้ต่อบริบท (Context-Aware Neural Networks) อย่างแท้จริง

ผลสำเร็จของการประยุกต์ใช้งานวิเคราะห์ความรู้สึกเชิงบวก เชิงลบ หรือเป็นกลางนี้ ไม่เพียงแต่เป็นตัวแทนพิสูจน์ให้ประจักษ์ว่าคณิตศาสตร์สามารถสกัดรูปลักษณ์ทางตรรกะออกจากอารมณ์ความรู้สึกอันกำกวมได้ แต่กระบวนการปรับทูนทางวิศวกรรมข้อมูล ตลอดจนกรอบคิดในการวิพากษ์ประสิทธิภาพผ่านมุมมอง F1-Score ยังถือเป็นสารตั้งต้นสำคัญและเป็นองค์ความรู้ภาคบังคับ (Prerequisite) ก่อนที่ผู้เรียนจะ

ก้าวข้ามขีดจำกัดไปสู่สถาปัตยกรรมปฏิวัติวงการเทคโนโลยีขั้นสุดยอดอย่าง โครงสร้างแรงบันดาลใจตัวเองและ
โมเดลทรานสฟอร์มเมอร์ (Attention Mechanisms and Transformer Architecture) รวมทั้งโลกแห่งการ
พัฒนาระบบตัวแบบภาษาขนาดใหญ่ (Large Language Models: LLMs) ต่อไปในอนาคต

อ้างอิง

- Chantwanukul, P., & Vateekul, P. (2020). Thai sentiment analysis via bidirectional LSTM-CNN model with embedding vectors and sentic features. *ResearchGate*.
https://www.researchgate.net/publication/332526260_Thai_Sentiment_Analysis_via_Bidirectional_LSTM-CNN_Model_with_Embedding_Vectors_and_Sentic_Features
- Hasan, M. M., et al. (2023). Sentiment analysis challenges and insights. *Journal of Management and Science Research*. <https://www.jmsr-online.com/article/sentiment-analysis-challenges-and-insights-410/>
- Khomsah, A., & Ramadhani, R. (2022). The accuracy comparison between Word2Vec and FastText on sentiment analysis of hotel reviews. *ResearchGate*.
https://www.researchgate.net/publication/361829612_The_Accuracy_Comparison_Between_Word2Vec_and_FastText_On_Sentiment_Analysis_of_Hotel_Reviews
- National Institutes of Health. (n.d.). Article PMC9944441. PubMed Central.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC9944441/>
- PeerJ. (2025). *Computer Science*. <https://peerj.com/articles/cs-2592/>
- Prakobtham, P., et al. (2023). An efficient deep learning for Thai sentiment analysis. *ResearchGate*.
https://www.researchgate.net/publication/370783554_An_Efficient_Deep_Learning_for_Thai_Sentiment_Analysis
- Prasad, R. (2021). RANLP. *ACL Anthology*. <https://aclanthology.org/2021.ranlp-1.31.pdf>
- Saeed, M., et al. (2025). *Pakistan Journal of Life and Social Sciences*.
https://pjlss.edu.pk/pdf_files/2025_1/65-77.pdf
- Semanticscholar. (2022). The accuracy comparison between Word2Vec and FastText on sentiment analysis of hotel reviews. <https://www.semanticscholar.org/paper/The-Accuracy-Comparison-Between-Word2Vec-and-On-of-Khomsah-Ramadhani/5567447543227d92658cd4223eda316dbef82346>
- Tiwari, P., et al. (2022). MDPI Electronics, 11(12), 1906. <https://www.mdpi.com/2079-9292/11/12/1906>

Wang, Y., et al. (2024). MDPI Applied Sciences, 14(23), 11435. <https://www.mdpi.com/2076-3417/14/23/11435>

Zhou, X., et al. (2024). ArXiv. <https://arxiv.org/pdf/2404.14631>

Zhou, X., et al. (2025). ArXiv. <https://arxiv.org/html/2502.20682v1>

Chulalongkorn University. (n.d.). *Electronic Thesis and Dissertation*. CUIR.

<https://digital.car.chula.ac.th/cgi/viewcontent.cgi?article=5654&context=chulaetd>

Durna, M. B. (n.d.). *Advanced word embeddings: Word2Vec, GloVe, and FastText*. Medium.

<https://medium.com/@mervebdurna/advanced-word-embeddings-word2vec-glove-and-fasttext-26e546ffedbd>

Ganesan, K. (n.d.). *Comparison between CBOW, Skip-Gram, and subword models*.

<https://kavita-ganesan.com/comparison-between-cbow-skipgram-subword/>

Gayanuka. (2025). *Word2Vec galaxy*. <https://gayanukaa.com/blog/2025-07-18-word2vec-galaxy>

GeeksforGeeks. (n.d.). *Deep learning: Introduction to Long Short Term Memory*.

<https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>

GeeksforGeeks. (n.d.). *Difference between a bidirectional LSTM and an LSTM*.

<https://www.geeksforgeeks.org/deep-learning/difference-between-a-bidirectional-lstm-and-an-lstm/>

GeeksforGeeks. (n.d.). *Metrics for machine learning model*.

<https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>

GeeksforGeeks. (n.d.). *Word embeddings in NLP: Comparison between CBOW and Skip-gram models*.

<https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp-comparison-between-cbow-and-skip-gram-models/>

Ido, L. (n.d.). *Skip-gram Word2Vec algorithm explained*. Medium. [https://leshem-](https://leshem-ido.medium.com/skip-gram-word2vec-algorithm-explained-85cd67a45ffa)

[ido.medium.com/skip-gram-word2vec-algorithm-explained-85cd67a45ffa](https://leshem-ido.medium.com/skip-gram-word2vec-algorithm-explained-85cd67a45ffa)

- Kashyap, P. (n.d.). *Understanding precision, recall, and F1 score metrics*. Medium.
<https://medium.com/@piyushkashyap045/understanding-precision-recall-and-f1-score-metrics-ea219b908093>
- Leyaa AI. (n.d.). *NLP multiclass text classification: Complexity*. Codefly.
<https://leyaa.ai/codefly/learn/nlp/part-1/nlp-multiclass-text-classification/complexity>
- ML Review. (n.d.). *Understanding LSTM and its diagrams*.
<https://blog.mlreview.com/understanding-lstm-and-its-diagrams-37e2f46f1714>
- Olah, C. (2015, August 27). *Understanding LSTMs*. Colah's Blog.
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- PyImageSearch. (n.d.). *Word2Vec: A study of embeddings in NLP*.
<https://pyimagesearch.com/2022/07/11/word2vec-a-study-of-embeddings-in-nlp/>
- Ragoli86. (n.d.). *Word embeddings in NLP*. Dev.to. <https://dev.to/ragoli86/word-embeddings-in-nlp-5f0h>
- Towards Data Science. (n.d.). *NLP 101: Word2Vec, Skip-gram, and CBOW*. Medium.
<https://medium.com/data-science/nlp-101-word2vec-skip-gram-and-cbow-93512ee24314>
- Dive into Deep Learning. (n.d.). *Long Short-Term Memory (LSTM)*. D2L.ai.
https://d2l.ai/chapter_recurrent-modern/lstm.html
- Evidently AI. (n.d.). *Multi-class metrics*. <https://www.evidentlyai.com/classification-metrics/multi-class-metrics>
- Google Developers. (n.d.). *Classification: Accuracy, precision, and recall*. Machine Learning Crash Course. <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall?hl=th>
- PyThaiNLP. (n.d.). *API tokenize*. Dev Docs. <https://pythainlp.org/dev-docs/api/tokenize.html>
- Stack Exchange (Artificial Intelligence). (2018). *What are pros and cons of Bi-LSTM as compared to LSTM?* [Online Q&A].
<https://ai.stackexchange.com/questions/18759/what-are-pros-and-cons-of-bi-lstm-as-compared-to-lstm>